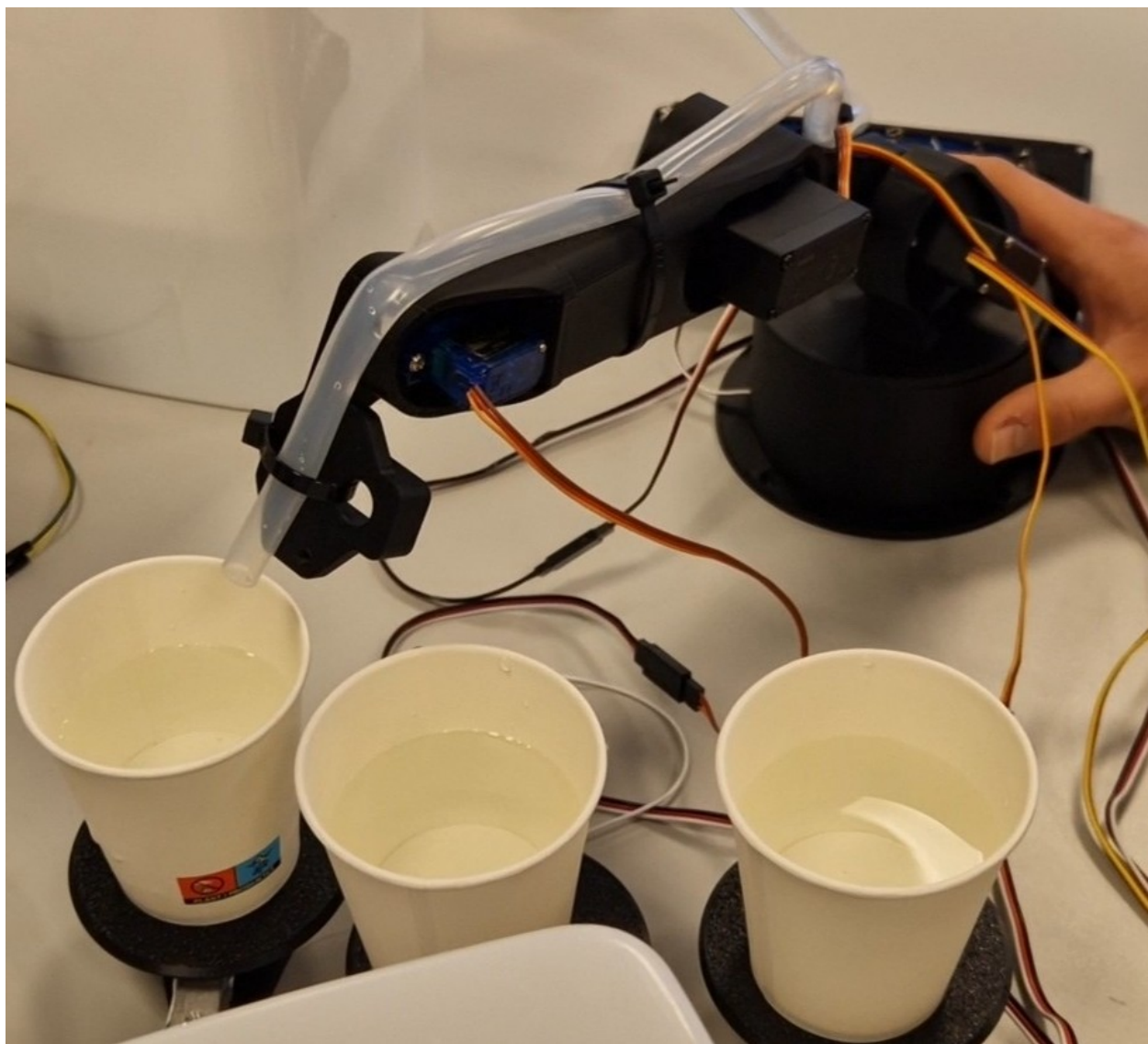


Fakultet for ingeniørvitenskap og teknologi
AUT-2606 Semester Prosjekt: Rapport

Automatisk refill arm

Institutt for Automasjon og Prosessteknologi
29.05 2026



Innhold

1	Sammendrag	1
2	Introduksjon	1
3	Teori	1
3.1	Raspberry Pi	1
3.2	PCA9685	1
3.2.1	I2C	1
3.3	HX711	2
3.3.1	Bit bang	2
3.4	Dual H-Bridge L298N module	2
3.5	Regulering	2
3.6	Invers kinematikk	3
4	Metode	4
4.1	Testing før mottatt PCB	4
4.2	Kretsen	4
4.3	Beste praksis for PCB-design	6
4.4	Programmering	6
4.4.1	PCA9685	6
4.4.2	HX711	6
4.5	main.py	7
4.5.1	Importerte biblioteker	8
4.5.2	Servostyring	8
4.5.3	Tilstand	8
4.5.4	Invers kinematikk	9
4.5.5	Jevn bevegelse	9
4.5.6	PID-regulator	9
4.5.7	Fylleprosess	10
4.5.8	Hovedløkke	10
4.5.9	GUI	11
4.5.10	Avslutning	12
5	Resultat	12
5.1	PCB	12
5.2	Arm	13
6	Diskusjon	14
7	Konklusjon	15
8	Kildehenvisning	16
9	Vedlegg	17

Figurer

1	PID sløyfe	2
2	Diagram over robotarmens geometri med tre frihetsgrader.	3
3	3D model av robot arm	4
4	Transistor oppsett	6

5	Package Diagram	7
6	Tilstandsdiagram	9
7	GUI	12
8	Hoved PCB og HX711 PCB	13
9	Endringene som ble gjort på PCB-en	13
10	Se vedlegg mappe for videoen	17
11	Deployment Diagram	17
12	Use-Case Diagram	17
13	Hoved kretskjema der alle individe komponenter ble satt sammen.	18
14	Pi interface som inkludere PCA9685, PWM switchers, og knapper	19
15	Hoved PCB	19
16	Individ Hx711 krets	20
17	HX711 PCB	20

Tabeller

1	Komponentliste (BOM) for PCB-ene.	5
---	---	---

Kodeutdragsliste

1	HX711 avlesing Kodeutdrag	7
2	Servo styring Kodeutdrag	8
3	Bevegelse Kodeutdrag	9
4	PID regulering Kodeutdrag	10
5	Hovedløkke Kodeutdrag	11
6	Avsluttning Kodeutdrag	12

1. Sammendrag

Vi har satt oss fore å lage en maskin som viser vår kunnskap om mekanismer, PCB-design og programmering. Hovedsystemet er satt sammen av en Raspberry Pi, tilpassede PCB-er, 3D-printede deler og andre elektronikkomponenter. Vi har tilpasset alle komponenter slik at de passer sammen, og designet PCB-ene for å levere den nødvendige funksjonaliteten: PWM-utganger, I/O-knapper, HX711 loadcell-brikker og dedikerte I2C-tilkoblinger. Programvaren gir PID-regulering, IK-posisjonering og tilstandsmaskiner. Til slutt har vi bygget, testet, feilrettet og ferdigstilt en robotarm som fyller glass når vektsensoren registrerer at de er tomme. Vi har dermed nådd målet for semesterprosjektet.

2. Introduksjon

I dette semesterprosjektet har vi laget en automatisk refill-arm, en robotarm som fyller glass med væske når en vektsensor registrerer at de er tomme. Prosjektet er gjennomført i emnet AUT-2606 ved Institutt for Automasjon og Prosessteknologi, og målet har vært å vise og videreutvikle vår kunnskap innen mekanismer, PCB-design og programmering.

Rapporten gir først en gjennomgang av det teoretiske grunnlaget for komponentene og metodene som er brukt, blant annet regulering og invers kinematikk. Deretter beskrives metodikken bak design av kretsen, lodding og programvaren, før vi presenterer og diskuterer resultatene fra det ferdige systemet.

3. Teori

3.1. Raspberry Pi

Raspberry Pi er datamaskinen i dette prosjektet. Det ble valgt for å bruke RPi fordi det er en kraftig datamaskin som kan kjøre programmer raskt, samtidig som den har GPIO-pinner som gir mulighet for tilkobling av ekstern tilkobling. RPi-en kan også kobles til GitHub, noe som gir en raskere og enklere arbeidsflyt. I tillegg støtter RPi-en alle kommunikasjonsprotokollene vi trengte i prosjektet, blant annet I2C og bit bang.

3.2. PCA9685

For å kunne kontrollere flere servomotorer samtidig ved bruk av RPi-en, brukte man chippen PCA9685. Denne gjør det mulig å styre opptil 16 servomotorer via I2C, med presise og stabile PWM signaler enn hva RPi-en kan gi.

3.2.1. I2C

I2C er kommunikasjon protokollen som blir brukt i PCA9685. I2C blir vanligvis brukt å koble mellom mikrokontrollere og sensorer, minnebrikker og andre enheter.

Buss strukturen til I2C er en synkron toveis kommunikasjonsbuss som kun benytter av to ledere: SDA (Serial Data) som er linjen som overfører data og SCL (Serial Clock) som er klokken som synkroniserer dataoverføringen laget av master enheten.

Kommunikasjonen bruker master og slave prinsipp. Det er en master enhet (vanligvis en mikrokontroller eller RPi i dette prosjektet) som styrer klokkesignalet og initialiserer kommunikasjonen. Og slave enheter svarer når kun de er adressert. Man kan tenke seg slikk at master enheten er posten som levere ut invitasjonen og slaven kan bare svare hvis de har fått invitasjoner. Hver slave har en unik adresse som er regel på 7 bit. Det gir i teori opp til 128 enheter på samme buss linje.

Data overføring skjer ved at, masteren sender en startbetingelse (SDA går lav mens SCL er høy). Masteren sender slavens 7-bits adresse. Den adresserte slaven svarer med et ACK-bit (acknowledge) for å bekrefte at den er tilstede. Data overføres byte for byte. Hvor hver byte følges av et ACK eller NACK bit fra mottakeren. Masteren avslutter med en stoppbetingelse (SDA går høy mens SCL er høy).

3.3. HX711

For å måle vekten på glassene brukte vi chippen HX711. Denne chippen leser av analoge signaler fra en lastcelle og konverterer dem til digitale data. HX711 er en 24 bits analog til digital omformer (ADC) spesielt beregnet for lastceller. Chippen kommuniserer med RPi-en via sin egen serielle protokoll basert på bit-banging. Siden hver HX711 kun kan lese fra en lastcelle, måtte vi ta i bruk tre chipper for å kunne lese av tre lastceller.

3.3.1. Bit bang

Bit banging er en teknikk der en kommunikasjonsmetode implementeres direkte i programvaren i stedet for ved hjelp av dedikert maskinvare. I stedet for å bruke allerede eksisterende bibliotek eller maskinvaremoduler i mikrokontrolleren, tolkes signalene ved at programvaren selv styrer en signallinje høy eller lav i riktig rekkefølge og med riktig timing.

Ved bit banging er det programvaren som tar seg av alle detaljene i protokollen. Dette går på å sette tilstanden på hver GPIO-pinne som er i bruk, manuelt. Å generere klokkesignalet ved å veksle mellom høyt og lavt. Å beholde timingen og håndtere rekkefølgen på bitene.

3.4. Dual H-Bridge L298N module

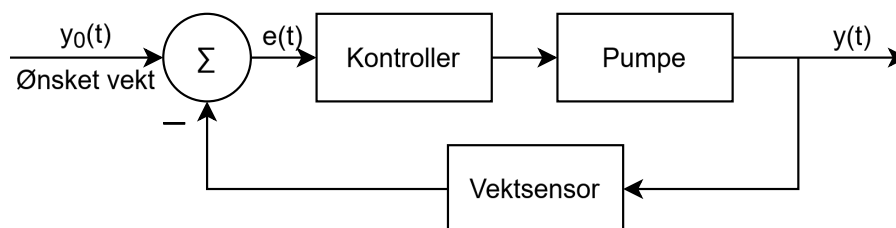
Modulen bruker både 5V og 12V. Den mottar et PWM-signal og har i tillegg en «HIGH-pin» og en «LOW-pin» som bestemmer hvilken retning pumpen går. Bytter man om på disse to pinnene, snur pumpe retningen.

Modulen er en dobbel bidireksjonell H-bro. Det betyr at den kan styre to DC-motorer samtidig uavhengig av hverandre. En H-bro er en standard-krets av fire transistorer satt sammen i en form som ligner på bokstaven H, hvor motoren er plassert i midten. Med å aktivere diagonalt motsatte transistorer kan strømmen gå gjennom motoren, og å aktivere motsatte transistorer vil de gå motsatt vei.

PWM-signalet brukes til å styre hastigheten på motoren. PWM-en veksler raskt mellom «HIGH» og «LOW», og forholdet mellom "HIGHTid og LOWtid kalles «duty cycle». En høy duty cycle gir motoren høyere hastighet, mens en lav duty cycle gir lavere hastighet.

3.5. Regulering

For å fylle glasset og sikre at det blir fylt opp uten søl, brukes en form for regulering. En regulator sin oppgave er å styre et system slik at den målte $y(t)$ følger en ønsket referanseverdi så presist som mulig. I dette prosjektet styres pumpen ut fra hvor mye væske som skal være i glasset, slik at fyllingen stoppes på riktig tidspunkt.



Figur 1: PID sløyfe

Reguleringen er bygget opp som en *lukket sløyfe*. Det innebærer at den målte $y(t)$ kontinuerlig sammenlignes med referanseverdien, og at differansen mellom dem brukes til å bestemme pådraget. Feilsignalet er definert som differansen mellom referanseverdien $y_0(t)$ og målte ut verdien $y(t)$:

$$e(t) = y_0(t) - y(t) \quad (1)$$

Uten PID følger kontrolleren en på/av-logikk:

$$u(t) = \begin{cases} 1 & \text{hvis } e(t) > 0 \\ 0 & \text{hvis } e(t) \leq 0 \end{cases} \quad (2)$$

For å oppnå en jevnere og mer presis styring brukes en PID regulator. PID regulatoren består av: et proporsjonalt ledd (P), et integrasjons ledd (I) og et deriverende ledd (D).

Og under er K_p , K_i og K_d forsterkningskonstantene for henholdsvis proporsjonal, integral og derivasjons leddet. Disse må tunes inn for å gi ønsket oppførsel, der målet er en rask respons med lite oversving og liten stasjonær feil og unngå oscillasjon.

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3)$$

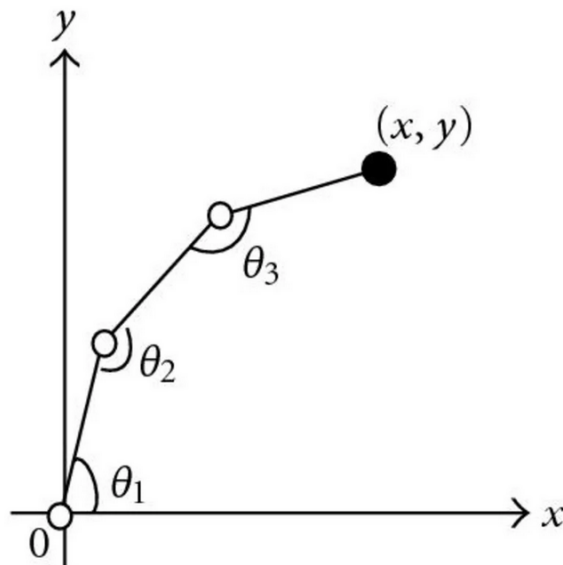
Den diskrete formelen som brukes i koden er:

$$u[k] = K_p \cdot e[k] + K_i \sum_{j=0}^k e[j] \cdot \Delta t + K_d \frac{e[k] - e[k-1]}{\Delta t} \quad (4)$$

Integralleddet er her tilnærmet med en Riemann sum, der feilen i hvert tidssteg multipliseres med Δt og summeres opp. Derivasjonsleddet er tilnærmet med baklengs differanse (finite difference), altså endringen i feilsignalet fra forrige til nåværende tidssteg delt på Δt . Disse tilnærmingene blir mer nøyaktige jo mindre Δt er.

3.6. Invers kinematikk

For å styre bevegelsen til robotarmen brukte vi en forenklet versjon av invers kinematikk. Vi valgte invers kinematikk fremfor fremoverkinematikk fordi fremoverkinematikk beregner endepunktet ut fra kjente leddvinkler, mens invers kinematikk gjør det motsatte den beregner nødvendige leddvinkler for å oppnå en ønsket endeposisjon. Robotarmens geometri er illustrert i figur 2.



Figur 2: Diagram over robotarmens geometri med tre frihetsgrader.

Definerer håndleddspunktet (x_w, y_w) ved å trekke fra bidraget fra det siste leddet med lengde L_3 :

$$x_w = x - L_3 \cos(\varphi) \quad (5)$$

$$y_w = y - L_3 \sin(\varphi) \quad (6)$$

Vinkelen θ_2 finnes ved hjelp av cosinussetningen:

$$\cos(\theta_2) = \frac{x_w^2 + y_w^2 - L_1^2 - L_2^2}{2L_1L_2} \quad (7)$$

$$\theta_2 = \text{atan2}\left(\pm\sqrt{1 - \cos^2(\theta_2)}, \cos(\theta_2)\right) \quad (8)$$

Deretter beregnes θ_1 og θ_3 :

$$\theta_1 = \text{atan2}(y_w, x_w) - \text{atan2}(L_2 \sin(\theta_2), L_1 + L_2 \cos(\theta_2)) \quad (9)$$

$$\theta_3 = \varphi - \theta_1 - \theta_2 \quad (10)$$

4. Metode



Figur 3: 3D model av robot arm

4.1. Testing før mottatt PCB

Siden produksjon og levering av PCB-ene tok tid, var det viktig å verifisere at de sentrale komponentene og kommunikasjonsprotokollene fungerte før kortene kom i posten. Raspberry Pi-en ble stilt til rådighet av universitetet tidlig i prosjektet, og vi brukte derfor denne til å teste de ulike delene av systemet hver for seg på forhånd.

Først testet vi I2C-kommunikasjonen, ettersom denne er avgjørende for styringen av servomotorene. Ved hjelp av en frittstående PCA9685-modul koblet til Raspberry Pi-en kunne vi bekrefte at Pi-en oppdaget enheten på I2C-bussen, og at vi klarte å generere stabile PWM-signaler ut fra modulen. På denne måten fikk vi verifisert både selve I2C-oppsettet og at vi kunne styre en servomotor via PCA9685 før dette ble integrert på vårt eget kort.

4.2. Kretsen

Vi tok hva vi hadde i POR-en for å finne ut hvilke funksjoner som var nødvendige, og hvilke som kunne inkluderes om vi hadde plass til det. Vi fant ut at kortet skal gi ut fem PWM-signaler, tre knapper, muligheten strømtilførsel uten USB-C, fire ledige pins for I2C-kommunikasjon, samt tre HX711-brikker for loadcells.

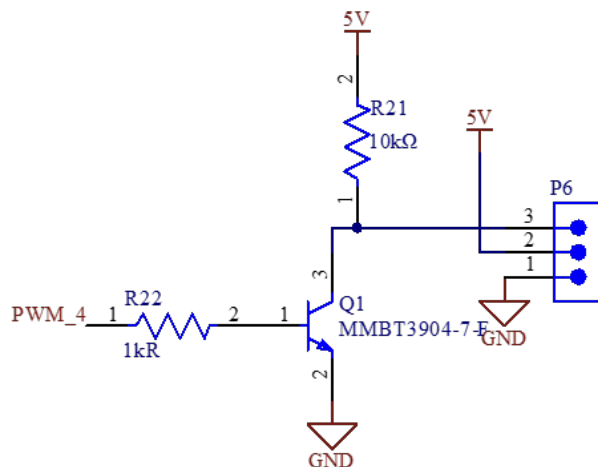
Tabell 1: Komponentliste (BOM) for PCB-ene.

Description	Designator	Mfr. Part Number	Supplier	Qty
CAP ALUM 10UF 20% 35V SMD	C1, C2, C5, C6, C7, C8	EDK106M035A9BAA	Farnell	6
CAP CER 0.1UF 25V X7R 0603	C3, C4, C9, C10, C11, C12	C0603C104K3RAC7411	Mouser	6
CAP CER 10UF 10V X5R 0603	C13	GRM188R61A106KE69D	Arrow	1
CC0603 100 nF X7R 10% 25 V	C14, C15, C16, C17	CC0603KPX7R8BB104	Mouser	4
LED GREEN CLEAR CHIP SMD	D2, D3	LTST-C193KGKT-5A	DigiKey	2
Female header, 40C, straight, 2.54 mm, PBT/phos bronze	J2	PPPC202LFBN-RC	DigiKey	1
CONN HEADER VERT 4POS 2.54MM	J3, J4, J5	TSW-104-23-T-S	Arrow	3
CONN HEADER VERT 6POS 2.54MM	J6	M20-9980346	DigiKey	1
SQ post socket, TH, vertical, 2.54 mm, 6-pin, female	J7	SSW-103-01-L-D	DigiKey	1
0.025SQ post header, TH, right-angle, 2.54 mm, 2-pin, male	P5	TSW-102-22-G-S-RA	Arrow	1
TRANS NPN 40V 0.2A SMD SOT23-3	Q1, Q2, Q3, Q8, Q9	MMBT3904-7-F	Farnell	5
Transistor PNP, 3-pin SC-75, Pb-free	Q5, Q6, Q7	MMBT3906TT1G	Mouser	3
RES thick film, 20 k Ω , 1%, 0.33 W, 0603	R1, R10, R13, R14, R17, R18	CRCW060320K0FKEAHP	Arrow	6
RES thick film, 10 k Ω , 1%, 0.2 W, 0603, AEC-Q200	R2, R4, R6, R7, R21	SG73P1JTTD1002F	Mouser	5
RES thick film, 1 k Ω , 1%, 0.1 W, 0603, AEC-Q200	R3, R5, R8, R9, R22	RK73G1JTTD1001F	Farnell	5
RES SMD 100 Ω 1% 1/10W 0603	R11, R12, R15, R16, R19, R20	RT0603FRE07100RL	Arrow	6
RES thick film, 2.2 k Ω , 5%, 0.1 W, 0603	R23	RC1608J222CS	Arrow	1
RES thick film, 3.3 k Ω , 5%, 0603	R24	CRGCQ0603J3K3	Arrow	1
SWITCH TACTILE SPST-NO 0.05A 24V	SW1, SW2, SW3	B3F-3122	Arrow	3

Ved bruk av Raspberry Pi-en ble hovedprosesseringen og styringen forenklet. Pi-en ga en fornuftig måte å kommunisere med de andre komponentene på, via I2C, bit-banging og innlesing gjennom GPIO-pinnene.

Først fant vi ut hvordan PWM-utgangene skulle styres. PWM-signalet genereres ikke direkte av Raspberry Pi-en, men av en PCA9685 PWM-driver som Pi-en kommuniserer med over I2C. Figur 4 viser hvordan transistoren brukes. Signalet fra PCA chippen føres til PWM inngangen i kretsen, der strømmen går gjennom en motstand som begrenser basestrømmen til transistoren. Transistoren er en NPN BJT som opererer i en «common-emitter»-konfigurasjon, det vil si at emitteren er felles for inngangen og utgangen. Deretter er collector-pinnen koblet til en pull-up-motstand.

Når PWM-signalet er høyt, driver basestrømmen transistoren til saturation. Collectoren blir da trukket ned mot GND, og utgangen settes lav. Når PWM-signalet er lavt, går det ingen strøm til basen, og transistoren er «av». Da går det ingen strøm gjennom pull-up-motstanden, og utgangen trekkes høy til 5V.



Figur 4: Transistor oppsett

Hx711 kretsen var en ukjent komponent som krevet mye forskning. Hoved krets design/oppsett kom fra en produsent som leverer ferdig bygget Hx711 PCB-er.

4.3. Beste praksis for PCB-design

Avkoblingskondensatorer: Hver aktive komponent ble utstyrt med avkoblingskondensatorer plassert så nær forsyningspinnene som mulig. PCA9685 og HX711-brikkene har dedikerte 100nF keramiske kondensatorer for å filtrere ut høyfrekvent støy, samt større 10 μ F elektrolytt-/keramiske kondensatorer for å stabilisere forsyningsspenningen ved raske lastendringer.

Jordplan: Kortet ble lagt ut med et sammenhengende jordplan (kobberfyll for GND). Et jordplan gir lav impedans tilbake til forsyningen og korte returbaner for signalstrømmene, noe som reduserer støy og gir mer stabile referansenivåer. Dette er særlig fordelaktig for de analoge målesignalene fra lastcellene, der et stabilt jordreferansenivå er avgjørende for nøyaktige avlesninger.

Sporbredde for forsyning: Spor som fører forsyningsstrøm (5V og 12V) ble tegnet merkbart bredere enn signalspor. Bredere spor har lavere resistans, noe som reduserer spenningsfallet og varmeutviklingen ved høyt strømtrekk. Dette er viktig i et system der servoer og pumpe kan trekke betydelig strøm samtidig.

Separasjon av analoge signaler: De analoge linjene fra lastcellene ble bevisst holdt adskilt fra PWM- og effektspor. Ved å unngå at følsomme målesignaler rutes parallelt med eller krysser støyende spor, reduseres innkobling av støy på HX711-avlesningene.

4.4. Programmering

4.4.1. PCA9685

PCA9685 chippen kommuniserer gjennom I2C og siden PCA9685 er en ganske populær chip som blir brukt mye gjennom hobby for å klare å styre flere servomotorer samtidig, så er det allerede en eksisterende bibliotek på python for denne chippen.

4.4.2. HX711

HX711 bruker sin egen type protokoll for kommunikasjon. Hvor RPi-en sender en 24 klokke pulser og for hver puls sender chippen ut et bit. Fra dette kan vi få data ut fra loadcellene som kan tolkes til vekt.

HX711-kretsen kommuniserer ikke over en standardisert maskinvarebuss, men bruker en egen toleders protokoll bestående av en datalinje (DT) og en klokkelinje (SCK). Denne protokollen er implementert i programvare ved bit banging. Innlesningen foregår slik: programmet venter først til datalinjen går lav, noe som signaliserer at kretsen har en ny måling. Deretter genereres klokkesignalet manuelt ved at SCK veksles høy og lav 24 ganger i en løkke. For hver klokkepuls leses en bit fra datalinjen og skiftes inn i et register. Til slutt sendes noen ekstra klokkepuls som velger kanal og forsterkning på kretsen, og hele resultatet på 24 bit settes sammen til ett tall som tolkes som et signert toerkomplement. Hele innlesningen er vist under:

```

1 def read_raw(self) -> int:
2     timeout = time.time() + 1.0
3     while self.dt.value == 1:
4         if time.time() > timeout:
5             raise TimeoutError("HX711 not responding, check wiring")
6     raw = 0
7     #Lock the shared clock so concurrent sensors dont corrupt each others reads
8     with sck_lock:
9         for _ in range(24):
10             sck.on()
11             raw = (raw << 1) | self.dt.value
12             sck.off()
13         for _ in range(GAIN_PULSES - 24):
14             sck.on()
15             sck.off()
16     if raw & 0x800000:
17         raw -= 0x1000000
18     return raw

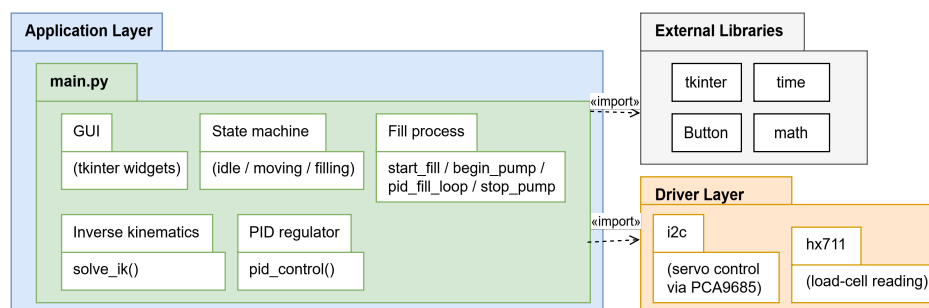
```

Kodeutdrag 1: HX711 avlesing Kodeutdrag

Til slutt sendes noen ekstra klokkepuls som velger kanal og forsterkning på kretsen. Hele resultatet på 24 bit settes så sammen til ett tall som tolkes som et signert toerkomplement. Siden flere sensorer i prosjektet deler den samme klokkelinjen, beskyttes klokkegenereringen av en lås `sck_lock` slik at to sensorer ikke kan flette seg inn i klokkepuls i hverandres innlesning samtidig.

4.5. main.py

Programmet styrer helle systemet samt og knytter sammen brukergrensesnittet (GUI), sensoravlesningen, servostyringen og pumpe reguleringen. Dette består av seks hoveddeler: import av nødvendige biblioteker, servostyring, invers kinematikk og PID-regulering, en tilstandsmaskin for fylle prosessen og en hovedløkke som henter information fra vektcellene.



Figur 5: Package Diagram

4.5.1. Importerte biblioteker

- `tkinter` - brukes til å lage brukergrensesnittet (GUI)
- `time` - brukes til tidsmåling i PID regulatoren
- `math` - brukes til trigonometriske beregninger i invers kinematikk
- `adafruit_servokit` - brukes til å styre servomotorene via PCA9685-driveren over I2C
- `hx711` - egen modul for avlesning av vektceller
- `gpiozero` - brukes til å styre GPIO-pinner, som pumpen og de fysiske tare knappene

4.5.2. Servostyring

Funksjonen `drive()` er den for å styre servoene på. Den tar inn fire leddvinkler (midje, skulder, albue, wrist) i grader samt en pumpe i prosent og oversetter disse til en duty cycle mellom 2350 og 5650. Disse to verdiene ble funnet etter noen forsøk der tallene ble justert mens PWM-signalet ble observert på oscilloskop. Dette området tilsvarer omtrent 800µs til 2200µs i pulsbredde. Verdiene sendes til PCA9685-driveren via `ServoKit`. Alle andre funksjoner i programmet som beveger armen bruker `drive()`.

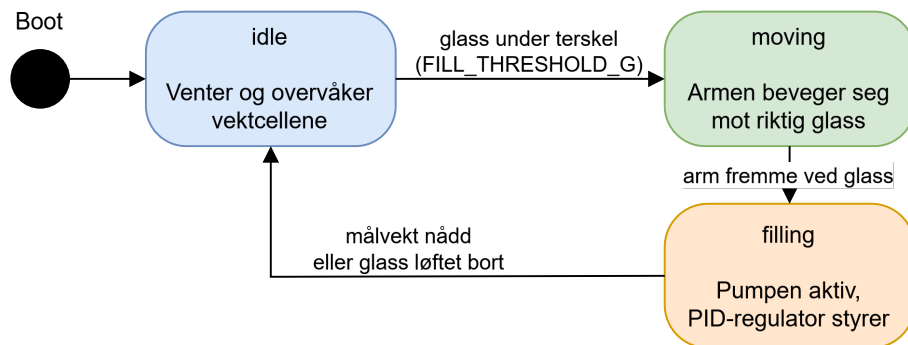
```
1 def drive(midje, skulder, albue, wrist, pump):
2     min = 2350
3     max = 5650
4
5     def angle_to_us(deg):
6         return int(max - ((max - min) * (deg / 90)))
7
8     kit._pca.channels[0].duty_cycle = 65535 - angle_to_us(midje)
9     kit._pca.channels[1].duty_cycle = 65535 - angle_to_us(skulder)
10    kit._pca.channels[2].duty_cycle = 65535 - angle_to_us(albue)
11    kit._pca.channels[3].duty_cycle = 65535 - angle_to_us(wrist)
12
13    desired_duty = int(pump / 100 * 65535)
14    kit._pca.channels[4].duty_cycle = 65535 - desired_duty
```

Kodeutdrag 2: Servo styring Kodeutdrag

4.5.3. Tilstand

Programmet bruker en global tilstandsmaskin med tre tilstander for fylleprosessen:

- `idle` - systemet venter og overvåker vektcellene
- `moving` - armen beveger seg mot riktig glass
- `filling` - pumpen er aktiv og PID-regulatoren styrer fyllingen



Figur 6: Tilstandsdiagram

4.5.4. Invers kinematikk

Funksjonen `solve_ik()` beregner leddvinklene som trengs for å sette armens tupp i ønsket posisjon (x, y, z) med ønsket orientasjon ϕ . Beregningen er basert på armens lengder L_1 , L_2 og L_3 , samt basehøyden Z_{base} fordi den ligger litt over nullpunktet $(0, 0, 0)$. Skuldervinkelen finnes ved trigonometri ut fra håndleddspunktet, mens albuvinkelen beregnes med cosinussetningen. Til slutt justeres vinklene for hvordan servoene er montert, slik at de kan sendes rett til servoene.

I den nåværende implementasjonen brukes `solve_ik()` kun når brukeren skriver inn koordinater i IK-feltet i GUI-et. Når armen kjører automatisk beveger armen seg i stedet til posisjoner som er lagret manuelt på forhånd.

4.5.5. Jevn bevegelse

Funksjonen `smooth_step()` beregner ut små delsteg mellom nåværende posisjon og målposisjonen over et fast antall steg (`MOVE_STEPS`, standard 40) med `MOVE_MS` millisekunder mellom hvert steg. Dette følger en *smoothstep*-kurve, $t^2(3 - 2t)$, slik at bevegelsen starter sakte, akselererer i midten og bremser mot slutten. Dette gir jevnere og mer kontrollert bevegelse enn en lineær interpolasjon.

```

1 def smooth_step(step):
2     t = step / MOVE_STEPS
3     t = t * t * (3 - 2 * t)
4     interp = [_move_start[i] + (_move_target[i] - move_start[i]) * t for i in
5               range(5)]
6     set_joints(interp)
7     if step < MOVE_STEPS:
8         root.after(MOVE_MS, lambda: smooth_step(step + 1))

```

Kodeutdrag 3: Bevegelse Kodeutdrag

4.5.6. PID-regulator

Funksjonen `pid_control()` bruker en diskret PID-regulator. Regulatoren styrer pumpeeffekten basert på forskjellen mellom ønsket vekt og målte vekt fra vektcellen. Standardverdiene til PID regulatoren er satt til:

- $K_p = 0,15$
- $K_i = 0,01$
- $K_d = 0,00$

Disse kan justeres direkte i GUI-en.

```
1 def apply_pid():
2     global kp, ki, kd, integral, prev_error, prev_time
3     kp = pid_kp_var.get()
4     ki = pid_ki_var.get()
5     kd = pid_kd_var.get()
6     integral = 0
7     prev_error = 0
8     prev_time = time.time()
9     pid_status.config(text=f"Active: Kp={kp} Ki={ki} Kd={kd}", fg="green")
```

Kodeutdrag 4: PID regulering Kodeutdrag

4.5.7. Fylleprosess

Fylleprosessen styres av fire funksjoner:

- `start_fill()` - setter tilstanden til `moving` og flytter armen til riktig glass via `go_to()`
- `begin_pump()` - aktiverer pumpen, nullstiller PID-tilstanden og setter tilstanden til `filling`
- `pid_fill_loop()` - kjører PID-regulatoren i en løkke med `PID_POLL_MS` intervall (standard 500 ms) til målvekten er nådd
- `stop_pump()` - slår av pumpen og returnerer tilstanden til `idle`

Hvis vektcellen leser en negativ verdi før eller under fylling tolkes dette som at glasset er løftet bort og avbrytes automatisk fyllingen.

4.5.8. Hovedløkke

Funksjonen `main_loop()` kjøres hvert 500ms og leser av alle tre vektcellene. Hvis systemet er aktivert via *Activate Main Program*-knappen og et glass er under terskelverdien `FILL_THRESHOLD_G` (standard 150g) vil fylleprosessen starte automatisk for det aktuelle glasset. Negative avlesninger ignoreres, slik at et glass som er løftet av vekten ikke starter fyllingen.

```

1 def main_loop():
2     if not activate or fill_state != "idle":
3         root.after(500, main_loop)
4         return
5     sensors = [hx711.sensor1, hx711.sensor2, hx711.sensor3]
6     for idx, sensor in enumerate(sensors):
7         try:
8             grams = sensor.read_grams()
9             if grams < 0:
10                 continue #løftet glass
11             if grams < FILL_THRESHOLD_G:
12                 start_fill(idx) #fyll glass
13                 return
14         except Exception:
15             pass
16     root.after(500, main_loop)

```

Kodeutdrag 5: Hovedløkke Kodeutdrag

4.5.9. GUI

GUI-et er bygget med `tkinter` og inneholder:

- **Joint Control** - glidere for manuell styring av hvert ledd (midje, skulder, albue, wrist) samt pumpe
- **Inverse Kinematics** - inntastingsfelt for målposisjon (x, y, z, ϕ) og geometriparametere
- **Saved Positions** - lagring og avspilling av inntil tre posisjoner med jevn interpolasjon
- **Load Cell** - sanntidsavlesning av alle tre vektceller med valg mellom live og en median modus
- **Tare og Kalibrering** - nullstilling og kalibrering av vektcellene, både via GUI og via fysiske knapper koblet til GPIO
- **Fill Settings** - innstilling av terskel og målevekt for fylling
- **Water Level** - fremdriftsindikatorer som viser fyllingsnivå for hvert glass
- **PID Controller** - felt for K_p , K_i og K_d med knapp for å anvende nye verdier under kjøring



Figur 7: GUI

4.5.10. Avslutning

Når programmet avsluttes frigjøres det alle ressurser:

```

1  hx711.sensor1.close()
2  hx711.sensor2.close()
3  hx711.sensor3.close()
4  hx711.sck.close()
5  pump_fwd.close()

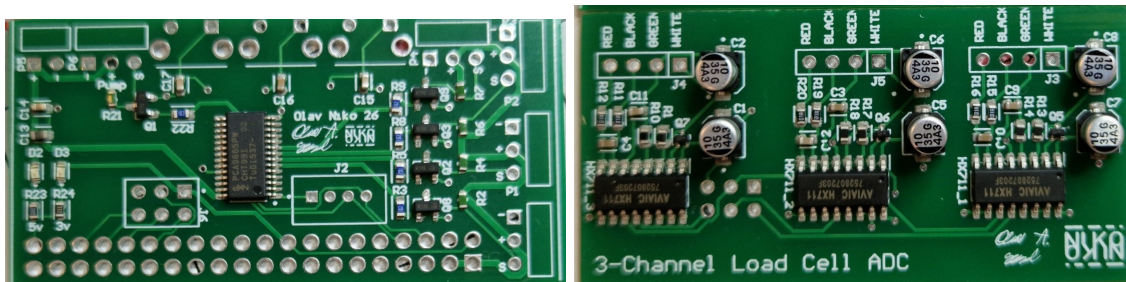
```

Kodeutdrag 6: Avslutning Kodeutdrag

5. Resultat

5.1. PCB

Ved testing av PCB-en viste tre punkter som måtte justeres før kortet nådd full funksjonaleitet. PCB-en fungerte ellers som planlagt, og justeringer kunne utføres direkte på kortet uten å bestille på nytt.



Figur 8: Hoved PCB og HX711 PCB

Det første punktet var at OE-pinnen på PCA9685 ble latt flytende. Pinnen aktiveres når den er lav, ikke var koblet til jord. Dette løste vi ved å lodde en bru mellom den pinnen og en nærliggende pinne som var koblet til jord.

Det andre punktet var at den SDA ledningen på PCB-en var ikke koblet til SDA pinnen på PI-en. Dette kunne i prinsippet løses ved å konfigurere den ikke-dedikerte pinnen for I2C gjennom programvaren på RPi-en, men siden denne pinnen manglet pull-resistor, valgte vi i stedet å bygge om forbindelsen med en kabel.

Det tredje punktet var at styringen av pumpe motoren krevde en pinne som kunne settes både lav og høy for å aktivere og deaktivere H-broen. Dette ble løst ved å lodde på en ekstra pinne.

Etter at disse justeringene ble gjort, verifiserte vi kortet ved hjelp av enkle testprogrammer som kjører en funksjon om gangen. Deretter kunne vi bekrefte at alle delkomponenter av PCB-en fungerte som de skal.



Figur 9: Endringene som ble gjort på PCB-en

5.2. Arm

Testingen ble gjennomført trinnvis, der de ulike delene av systemet først ble verifisert hver for seg. De tre forhåndsbestemte posisjonene ble testet for å bekrefte at armen klarte å bevege seg mellom de lagrede posisjonene. Deretter ble det verifisert at pumpen startet når loadcellen registrerte at vekten kom under terskelverdien, og stoppet når vekten kom over ønsket mengde, samt at pumpen ikke startet dersom koppen ble tatt av brikken. Til slutt ble systemet testet med vann. Da delfunksjonene ble kombinert, fungerte de sammen som en tilstandsmaskin: tilstandsmaskinen styrer overgangene mellom å bevege armen, fylle glasset og returnere til hviletilstand, mens PID-regulatoren sørger for at fyllingen stoppes ved riktig målvekt uten søl.

Under testingen observerte vi to kontinuerlige avvik. For det første hadde armen i sjeldne tilfeller problemer med å nå posisjonene nøyaktig etter å ha beveget seg til en ny posisjon, og dette skjedde først etter lengre tids bruk. For det andre registrerte vi et konstant avvik i fyllemengden: glasset ble fylt med omtrent 12–20g vann mer enn målet av 100g. Dette skyldes at det fortsatt befinner seg vann i slangen i det øyeblikket pumpen slås av, slik at denne restmengden går ned i glasset etter at PID-regulatoren har stoppet pådraget.

Avviket var stabilt fra gang til gang, og kan derfor i prinsippet kompenseres for ved å justere målvekten tilsvarende ned. En til grunn for avviket kan være at Avlesningene fra HX711 chippene endret seg gradvis over tid selv uten endring i vekt.

Under testing av pumpen ble PID-verdiene justert, der Ki-verdien ble endret til 0.1. Reguleringen fungerte tilfredsstillende med denne ene endringen.

Under testingen sluttet servoen på skulderen å rotere etter å ha blitt utsatt for høyt stress. Ved nærmere undersøkelse viste det seg at de interne tannhjulene av plast var slitt ned. Servoen ble byttet ut med en servomotor med metallgir, og armen fungerte deretter som normalt.

Vi observerte også at RPi-en ga varsel om for lav strømtilførsel, og ved høyt strømtrekk oppstod det av og til "brownout", der RPi-en ble ustabil eller restartet. Dette inntraff særlig når servoene og pumpen var aktive samtidig. Problemet forsvant etter at de tynne, "daisy-chained"-kablene ble byttet ut med kraftigere kabler koblet direkte fra strømforsyningen til hoved-PCB-en.

Sammensatt har armen fungert som tiltenkt, der alle delfunksjoner kombinert førte til den automatiske påfyllingssekvensen. I vedlegg 10 kan man se hvordan påfyllingsprosedyren foregår.

6. Diskusjon

Resultatene viser at systemet fungerte etter hensikten. Armen klarte å registrere når en kopp kom under den gitte terskelverdien, bevege seg til riktig posisjon og fylle koppen til eller over målnivået.

Som nevnt i resultatene var det et avvik under fyllprosessen. Avviket vurderes som akseptabelt for fylling av et glass, men kan reduseres ved å forbedre målingene fra loadcellene. En feilkilde her var drift i verdiene fra HX711, der avlesningene endret seg gradvis over tid uten endring i vekt. Denne driften er en av årsakene til det varierende avviket etter fylling. Effekten av driften kan reduseres ved jevnlig nullstilling (tare), filtrering av avlesningene gjennom median- eller gjennomsnittsmodus, og bedre skjerming av kabler for å redusere støy.

At reguleringen fungerte tilfredsstillende med så lite tuning av PID-verdiene tyder på at de valgte standard-verdiene var godt egnet for systemet.

Likevel, selv om systemet nådde hensikten, var det noen svakheter og feilkilder i dette systemet.

En mekanisk svakhet var at de interne tannhjulene i servoene var laget av plast. Som beskrevet i resultatene førte dette til at servoen på skulderen sluttet å rotere etter høyt stress. Årsaken er at servoene som ble brukt ikke er dimensjonert for de kreftene armen utsettes for, noe som er en kjent svakhet ved billigere servoer med plastgir. Bytte til en servomotor med metallgir løste problemet, og er også å foretrekke på lengre sikt for å sikre armens presisjon og levetid. Det ble brukt to avkoblingskondensatorer på 5 V-skinen fra strømforsyningen, men lokal avkobling ved hver enkelt servo kunne potensielt redusert spenningstransienter ytterligere under høy belastning. Dette ble ikke testet, og er derfor kun en mulig forbedring.

Problemet med "brownout" på RPi-en skyldtes ikke selve strømforsyningen, men ledningene mellom strømforsyningen og hoved-PCB-en. Opprinnelig var det brukt tynne kabler som var "daisy-chained" videre til kortet, noe som ga for stort spenningsfall ved høyt strømtrekk, særlig når servoene og pumpen var aktive samtidig. En "brownout" kan føre til ustabil oppførsel eller at Pi-en restarter. Problemet ble løst ved å bytte til kraftigere kabler koblet direkte fra strømforsyningen til hoved-PCB-en. Dette reduserte spenningsfallet, og strømforsyningen leverte deretter stabil spenning også under høy belastning.

De tre feilene som ble oppdaget på hoved-IO-kortet lot seg fikse med loddebroer og kabling, og bør rettes opp i designet i en eventuell ny revisjon av kortet.

Noen tekniske valg ble gjort under dette prosjektet. Det ble valgt å bruke en egen PSU i stedet for en laboratorie strømforsyningen. Dette var fordi at systemet hadde behov for en strømforsyning som kunne levere nok amper til alle komponentene samtidig (kan se i POR-vedlegget). Det som er beskrevet over viste det seg likevel at strømforsyningen ikke var godt nok dimensjonert noe som førte til "brownout" ved høyt strømtrekk. Varselmeldingen på RPi-en om at det var for lite strøm kan være fordi PSU-en leverte 4,9V i stedet for 5V eller helst 5,1 V til RPi-en. Dette valget bør derfor revurderes med tanke på en annen PSU.

For å styre pumpen med PWM-signaler ble det valgt å bruke en ekstern modul (L298N) i stedet for å integrere H-broen direkte på PCB-en. Dette var fordi det begynte å bli lite plass igjen på PCB-en til alle funksjonene, og for å redusere risikoen for at hovedfunksjonen: å styre pumpen med PWM, ikke skulle fungere. Ved å legge pumpestyringen i en egen modul ble designet enklere og sjansen for feil på selve kortet redusert.

Samlet sett fungerte systemet etter hensikten. Pumpestyringen med PID-regulering ga presis fylling med kun et lite avvik, og robotarmen klarte å bevege seg mellom de forhåndsdefinerte posisjonene som planlagt. Svakheterne som ble avdekket lot seg alle utbedre underveis, og påvirket ikke det endelige resultatet.

7. Konklusjon

I dette prosjektet ble det laget en automatisk refill arm som fyller glass med væske når en vektsensor registrerer at de er tomme. Systemet ble bygget opp av en Raspberry Pi med egendesignede PCB-er, 3D-printede deler og øvrige elektronikkomponenter, og programvare som håndterer PID-regulering og invers kinematikk.

Prosjektet nådde hovedmålet sitt. Den ferdige armen klarte å registrere tomme kopper, bevege seg til riktig posisjon og fylle dem med et avvik på omtrent 10 g, noe som vurderes som tilstrekkelig presist for formålet.

For videre arbeid bør de tre feilene som ble oppdaget under testing — manglende jording av OE-pinnen på PCA9685, ombyttede I2C-pinner, og manglende styringspinne for pumpen — rettes direkte i designet i en ny revisjon av kortet, slik at de ikke lenger må utbedres med loddebroer og kabling. Videre bør servoene byttes ut med motorer med metallgir i stedet for plastgir for å gjøre armen mer robust. På lengre sikt kan systemet integreres med trådløs kommunikasjon mellom lastcellene og RPi-en, noe som ville gjort oppsettet mer fleksibelt og redusert behovet for kabling.

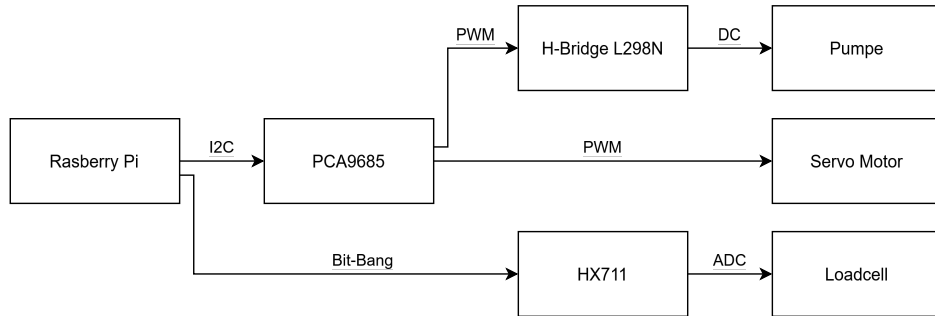
Et naturlig neste steg ville være å ta invers kinematikk fullt i bruk. I stedet for å lagre leddvinkler på forhånd kunne hvert glass hatt sine egne koordinater, og en IMU kunne registrert om glassene flyttes. På denne måten slipper man å lagre en posisjon manuelt for hver vektcelle, og systemet blir mer dynamisk. Slik koden fungerer nå, er IK i praksis redundant og fungerer mer som en ekstrafunksjon enn en nødvendig del av systemet.

8. Kildehenvisning

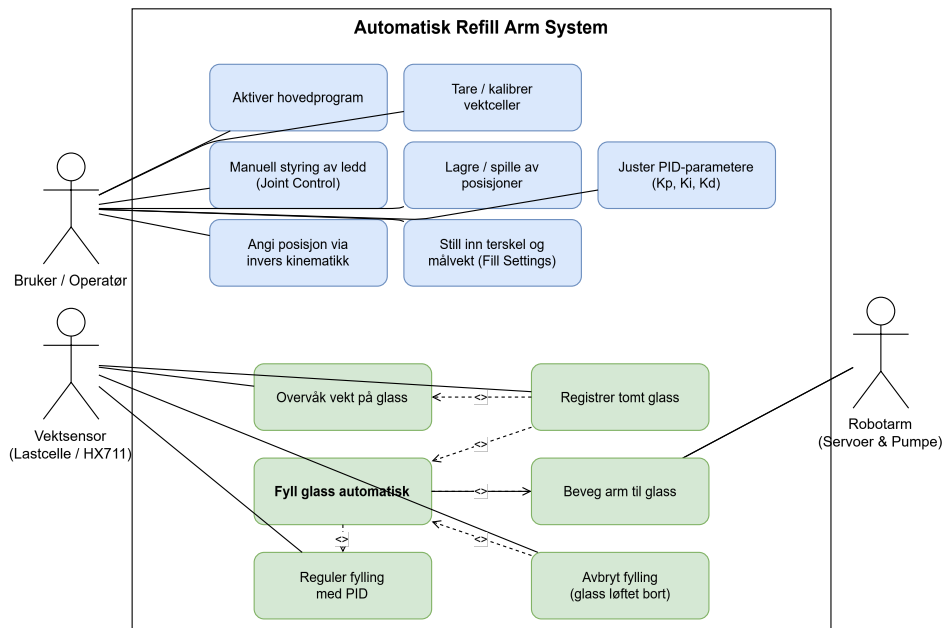
- Cults3D. (u.å.). *Arduino based robot arm – HowToMechatronics* [3D modell]. Hentet 26. mai 2026 fra <https://cults3d.com/en/3d-model/various/arduino-based-robot-arm-howtomechatronics>
- Vendelator. (u.å.). *Load Cell Scale Mount* [3D modell]. Printables. Hentet 26. mai 2025 fra <https://www.printables.com/model/374800-load-cell-scale-mount>
- Handson Technology. (u.å.). *L298N dual H-bridge motor driver* [Datablad]. Hentet 26. mai 2026 fra <https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf>
- NXP Semiconductors. (2015). *PCA9685: 16-channel, 12-bit PWM Fm+ I²C-bus LED controller* [Datablad]. <https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf>
- SparkFun Electronics. (u.å.). *HX711: 24-bit analog-to-digital converter (ADC) for weigh scales* [Datablad]. Hentet 26. mai 2026 fra https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf
- Wikipedia. (u.å.). *Inverse kinematics*. Hentet 26. mai 2026 fra https://en.wikipedia.org/wiki/Inverse_kinematics
- Ogawa, T. & Kanada, H. (2010). Inverse kinematics problem of 3-DOF robot arm in 2D plane [Figur og Invers kinematik]. I *Solution for Ill-Posed Inverse Kinematics of Robot Arm by Network Inversion*. ResearchGate. https://www.researchgate.net/figure/Inverse-kinematics-problem-of-3-DOF-robot-arm-in-2D-plane-a-Three-joints-and-an_fig3_46401997

9. Vedlegg

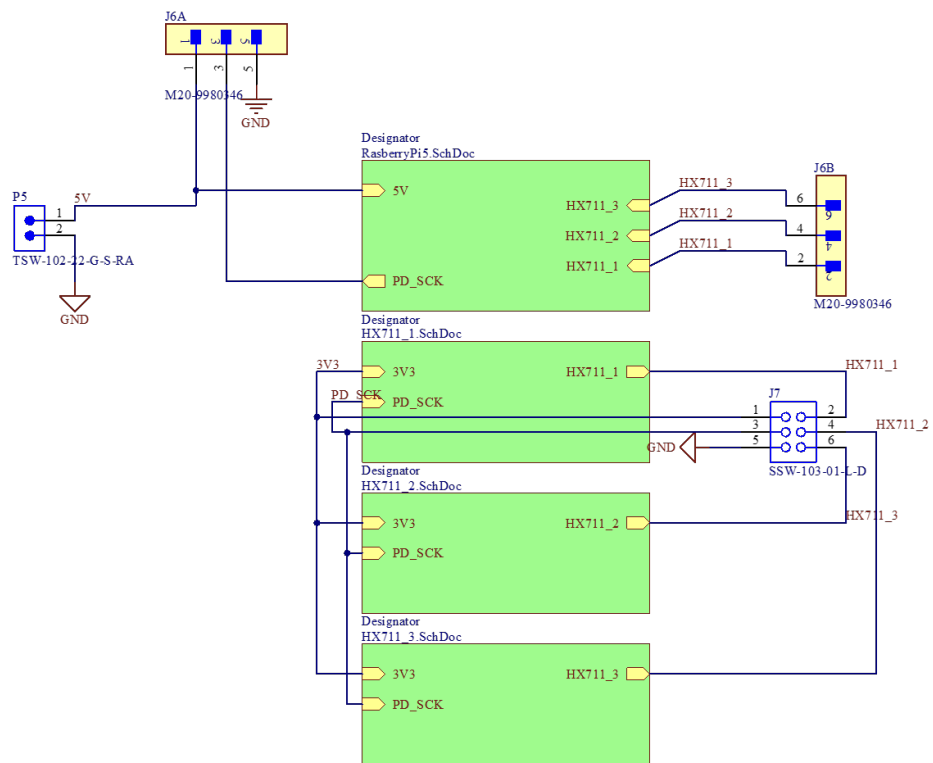
Vedlegg 10: Se vedlegg mappe for videoen



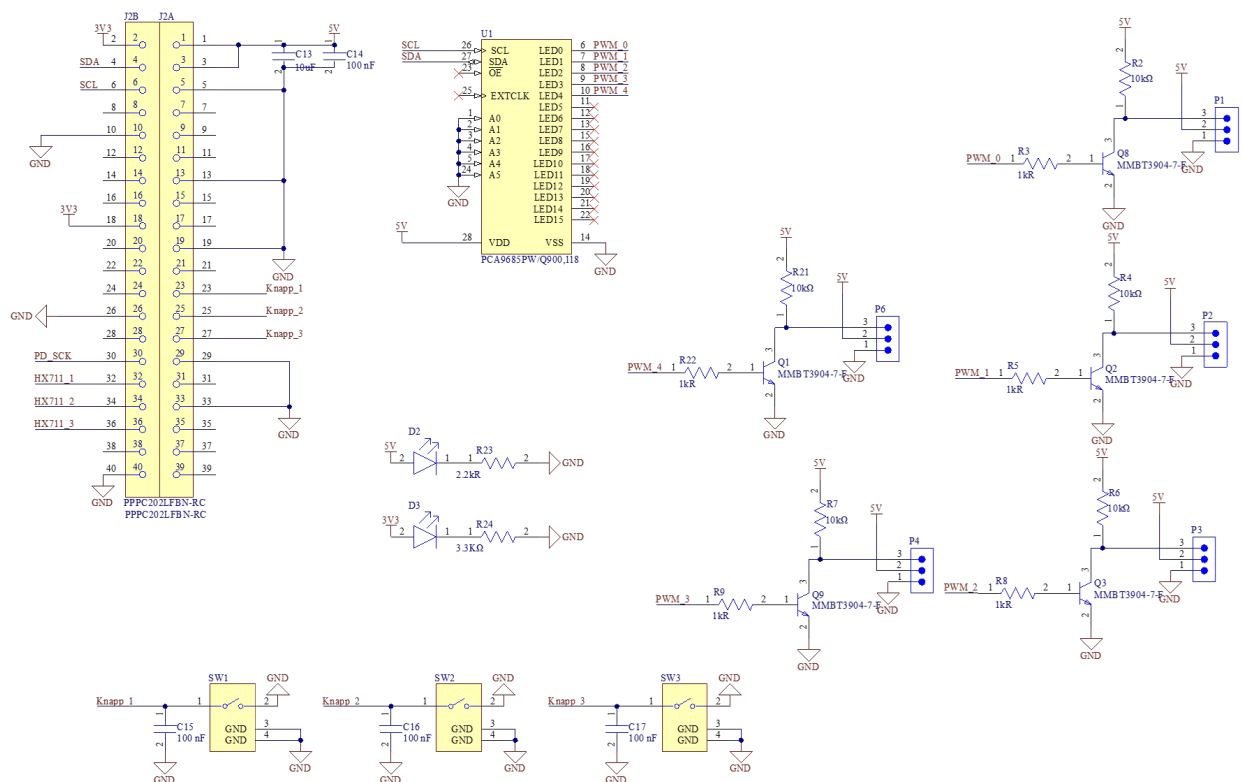
Figur 11: Deployment Diagram



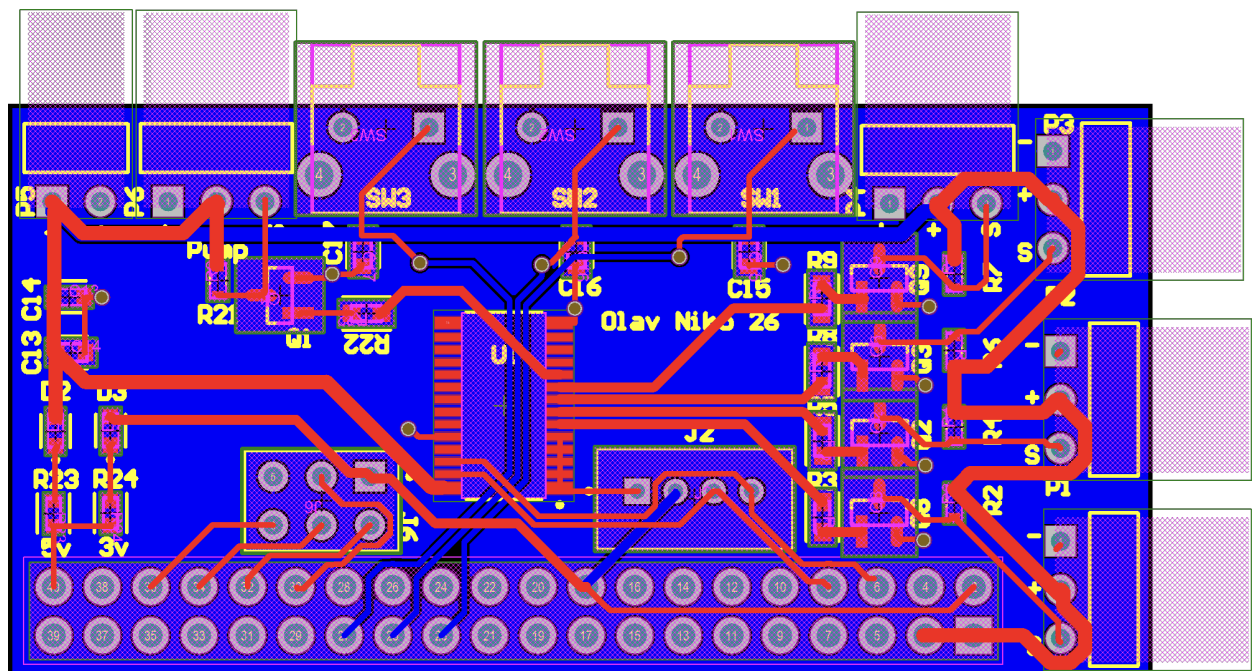
Figur 12: Use-Case Diagram



Figur 13: Hoved kretskjema der alle individe komponenter ble satt sammen.



Figur 14: Pi interface som inkluderer PCA9685, PWM switchers, og knapper



Figur 15: Hoved PCB

